AEGIS-NET

From network symptoms to audited remediation

Technical walkthrough: C analysis, anomaly detection, correlation, AI root-cause analysis, and a policy-gated DevSecOps agent

VERIFIED | UNVERIFIED HERE | SIMULATED | NOT IMPLEMENTED

Status tags used on the following slides

All data is synthetic. Prototype, not production. Repository commit 5d742e7 - 2026-10-20

THE PROBLEM

Troubleshooting a degraded network means stitching together scattered evidence

Evidence is scattered

- Metrics
- Deployments
- IDS alerts
- Logs

Four tools, four formats, one question: what changed?

Symptoms arrive in a cascade

9 alerts → 1 fault

memory, CPU, latency, jitter, loss, retransmits, drops, setup success, throughput

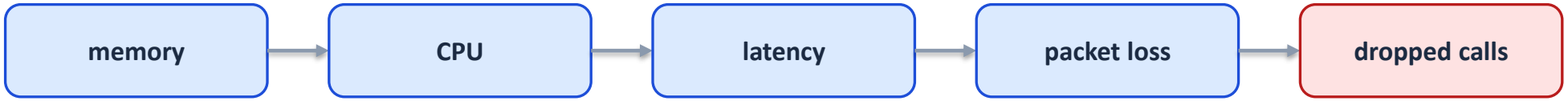
One fault, nine alerts. The first symptom is rarely the one people notice.

Decisions need an audit trail

- Rollback?
- Approval
- Record

A production rollback is risky and must be justified, approved and recorded.

The seeded incident as an operator sees it



Automated: evidence gathering, ordering, bookkeeping. **Still human:** judgement, approval, accepting risk.

Six capabilities, each with an honest status

C call analysis

Strict CSV parsing, per-cell and per-window statistics, robust outliers

VERIFIED

Telemetry and detection

17 KPIs, 9 incident types, 4 interpretable detectors

SIMULATED DATA

VERIFIED

Correlation and RCA

Ranked hypotheses; nine read-only tools; schema-validated answers

MOCK VERIFIED

LIVE LLM UNVERIFIED

Reports

JSON, Markdown, HTML computed from stored figures

VERIFIED

NO PDF

DevSecOps agent

Deterministic playbooks, risk classes, approvals, sandbox

VERIFIED

ROLLBACK SIMULATED

Platform

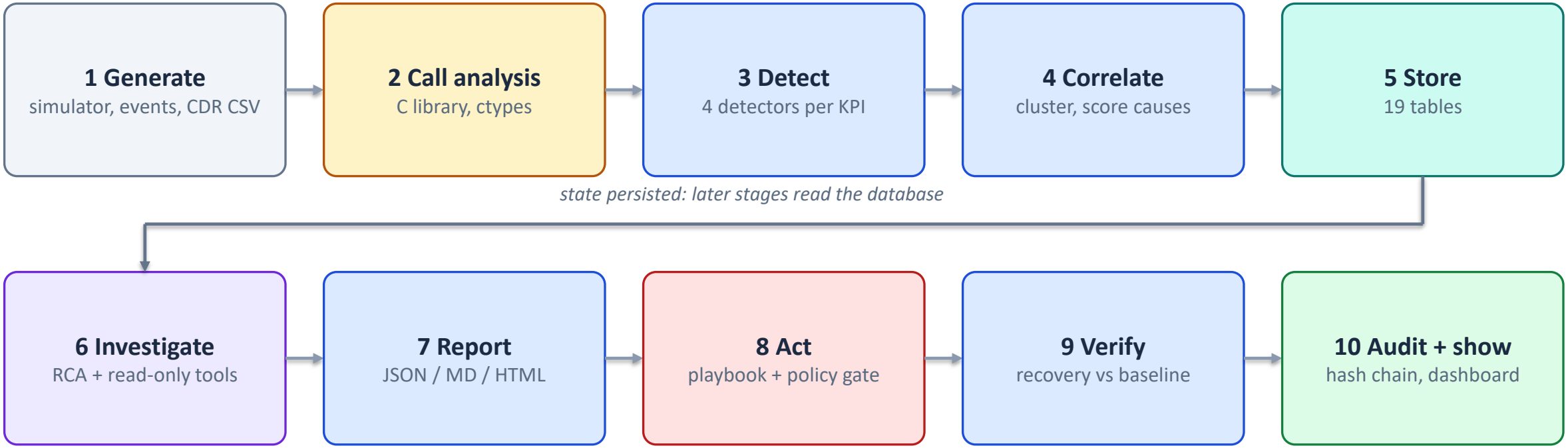
FastAPI, RBAC, hash-chained audit, React dashboard, Docker

VERIFIED

DOCKER NOT RE-RUN

END TO END

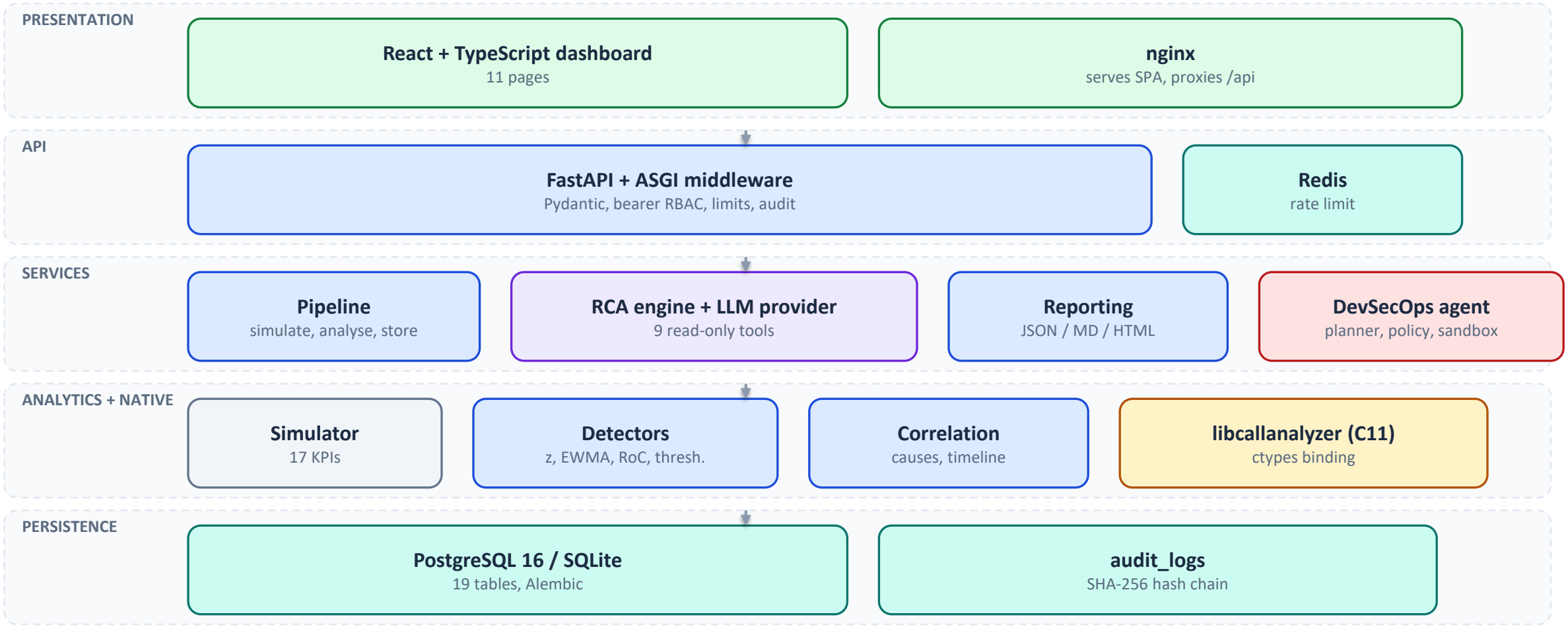
One incident travels through ten stages, and later stages read stored evidence



Recorded run: make demo (seed 7) - INC-2026-001 CRITICAL, 13 steps, exit 0, rollback verified, audit chain valid (24 rows).

ARCHITECTURE

Four languages meet at three explicit interfaces



Interfaces: C header, REST API (OpenAPI), database schema. Trust boundaries: CSV, LLM output, HTTP, agent to OS.

A C library parses and analyses call records; Python only marshals results

CDR CSV
8 columns

↓

libcallanalyzer
parse, aggregate, outliers

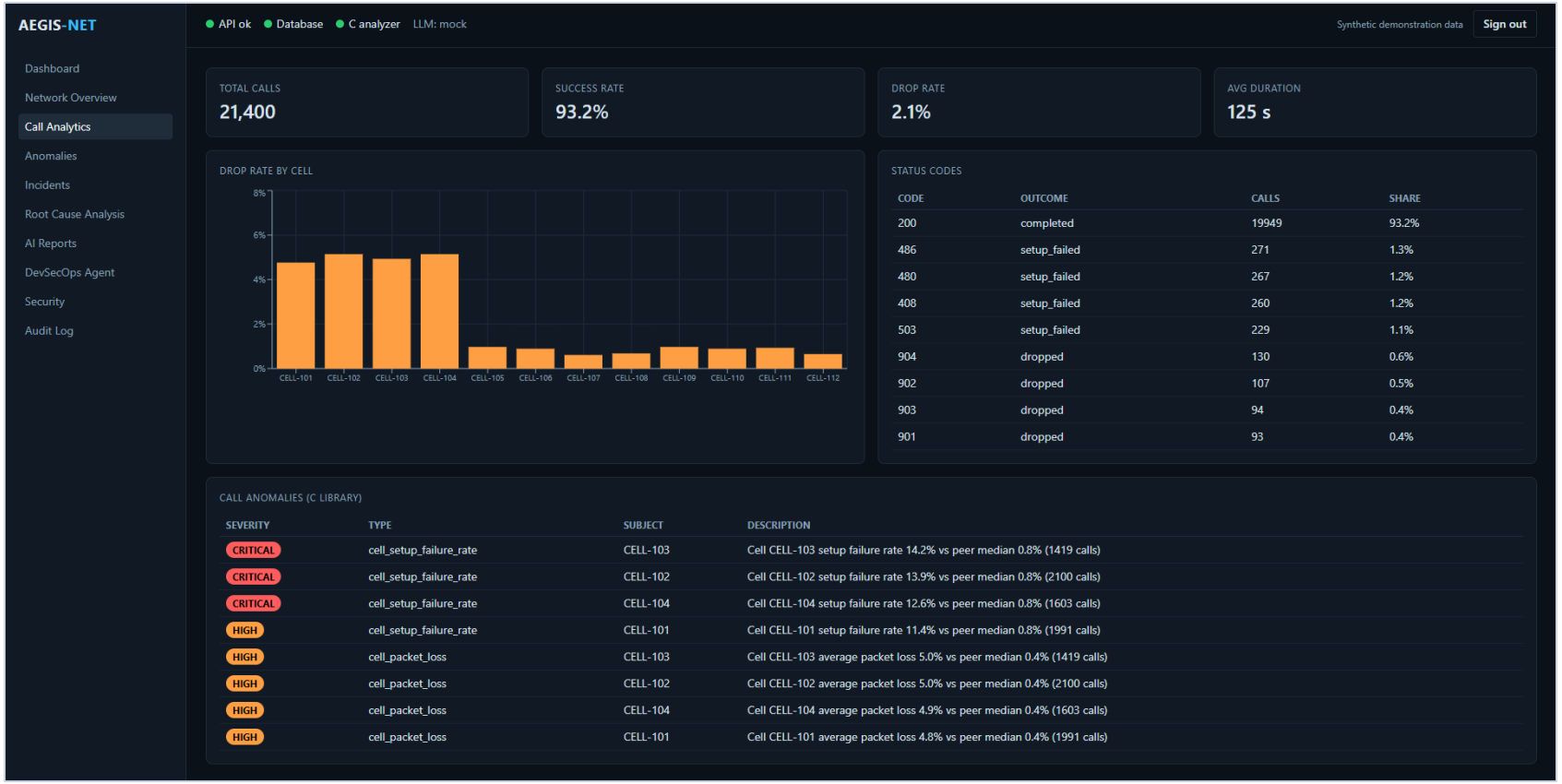
↓

ctypes binding
struct check, free_*

↓

Result dict
stats + anomalies

- strict warnings, -Werror
- ASan/UBSan/LSan + fault injection
- drop = dropped / (completed + dropped)



Screenshot: Call Analytics page (supplied document, synthetic data). Bars CELL-101 to 104 are the cells behind the faulty gateway. Status-code classes are this project's convention.

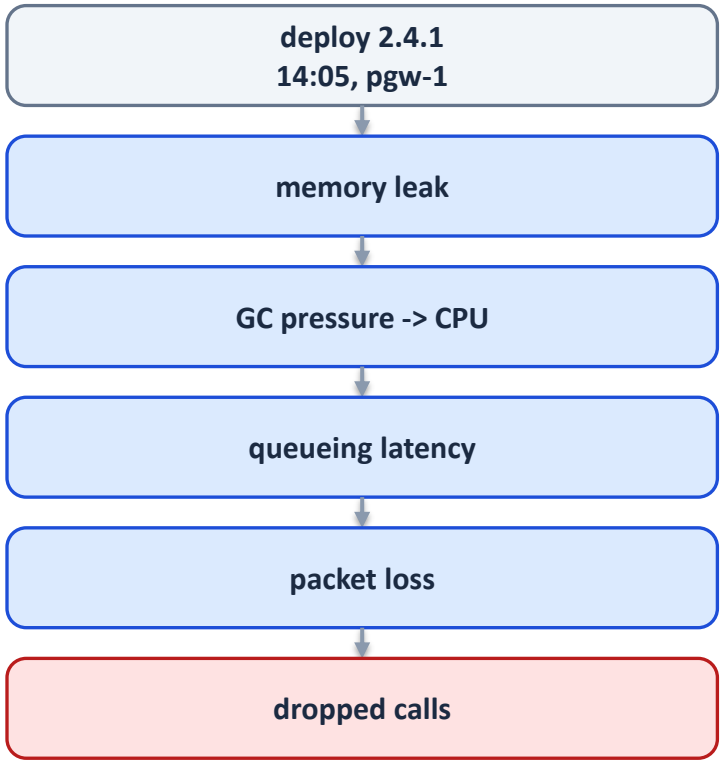
VERIFIED make c-test and make c-test-asan: 2 of 2 passed each; 28 test functions

A causal simulator produces a repeatable memory-leak incident



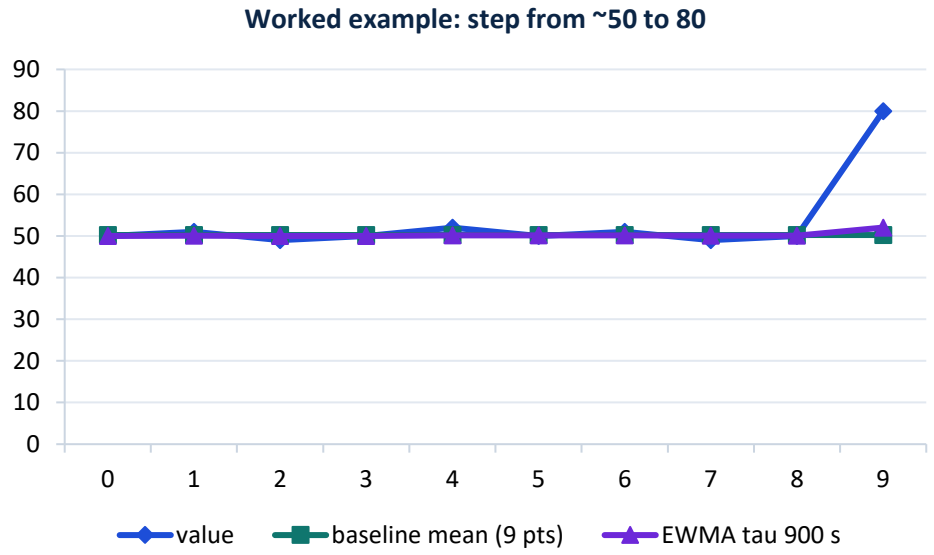
Screenshot: Network Overview (supplied document, synthetic data)

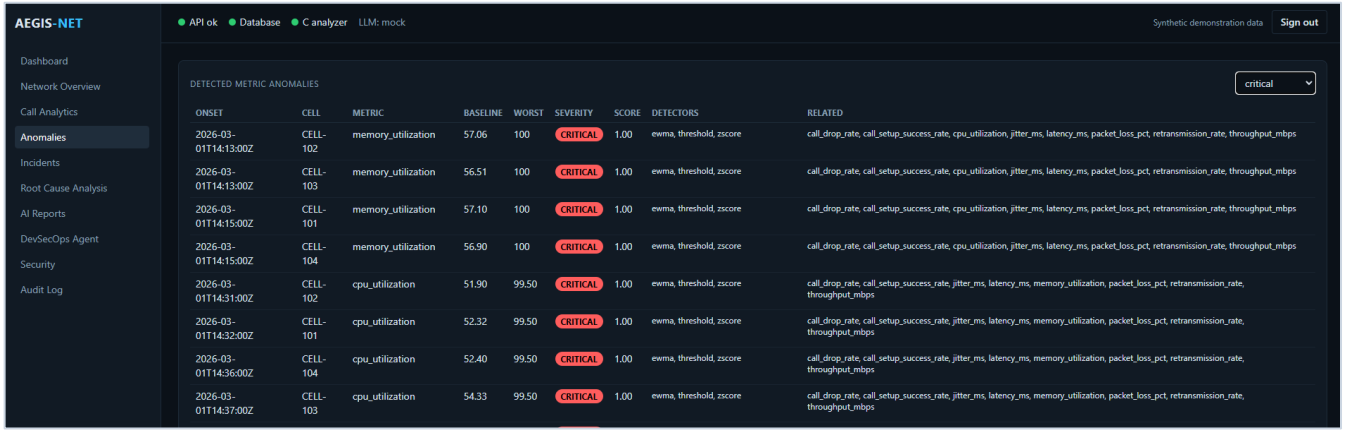
SIMULATED A testing instrument, not real traffic. Detector results here do not show how detectors would perform on production telemetry.



- 17 KPIs x 12 cells x 1 min, seed 7
- distractor: hotfix on pgw-3 at 13:10

Four interpretable detectors must agree before a point is flagged





z-score
(x - mean) / max(std, floor), 60 min window

EWMA
alpha = 1 - exp(-dt/tau), tau 900 s

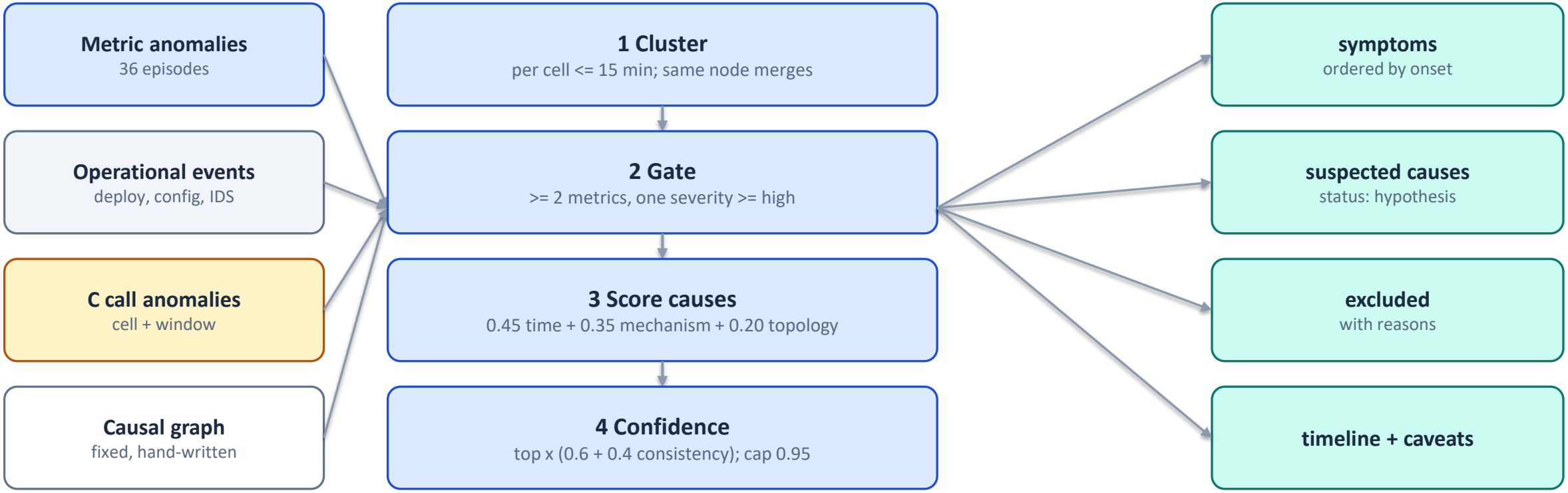
Threshold
warning / critical per KPI, e.g. latency 100 / 200 ms

Rate of change
per-minute change vs median / MAD, z >= 5

Flag: threshold OR (z AND EWMA) OR (RoC AND one), and the deviation must pass a practical minimum. **Score:** 1 - 2^(-z / threshold); 0.5 at the threshold. **Baseline:** only |z| < 2.5 points update it.

Chart data from the repository's own EWMA helper. Screenshot: Anomalies page, critical filter (supplied document, synthetic data).

The correlation engine turns many symptoms into one ranked, caveated incident

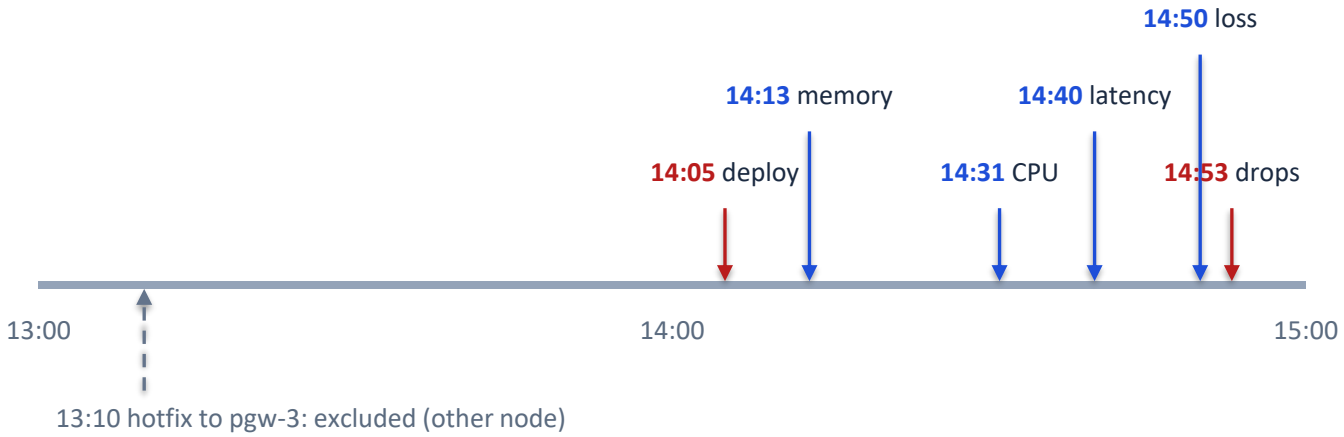


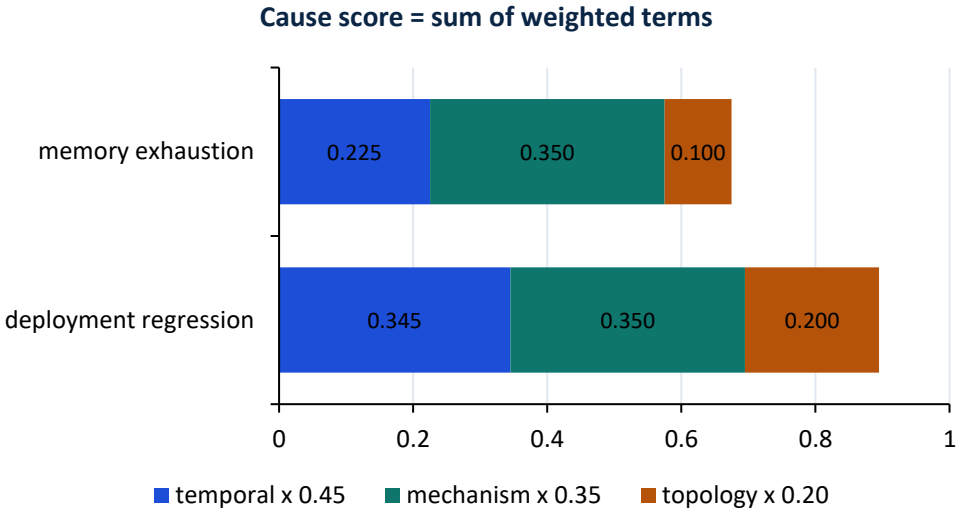
Correlation is not causation. The engine ranks hypotheses by time order, topology and a fixed causal graph; every cause carries status "hypothesis" and the incident carries a caveat to that effect.

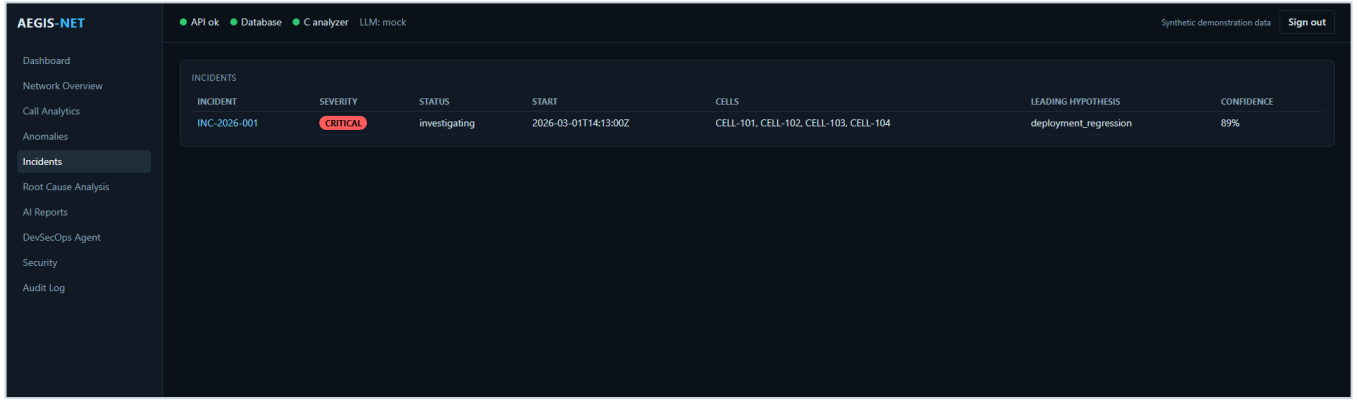
VERIFIED 29 tests; 0 incidents over 20 healthy days (recorded); seeded run recomputed for this handbook

The deployment outranks memory exhaustion because it preceded the first symptom on the same node

INC-2026-001 timeline (UTC, recomputed from seed 7)







Screenshot: Incidents page (supplied document). 89% is the correlation confidence; the RCA states 85% separately.

0.8947 deployment_regression

0.675 memory_exhaustion

- deployment was 8 min before onset: temporal exp(-8/30) = 0.766
- chain consistency 1.0, margin 0.22: confidence 0.8947

The model can only ask read-only tools; the engine validates every answer



Screenshot: Root Cause Analysis page, provider mock (supplied document, synthetic data)

MOCK VERIFIED **LIVE LLM UNVERIFIED**

12 tool calls in the seeded run; RCA confidence 85%

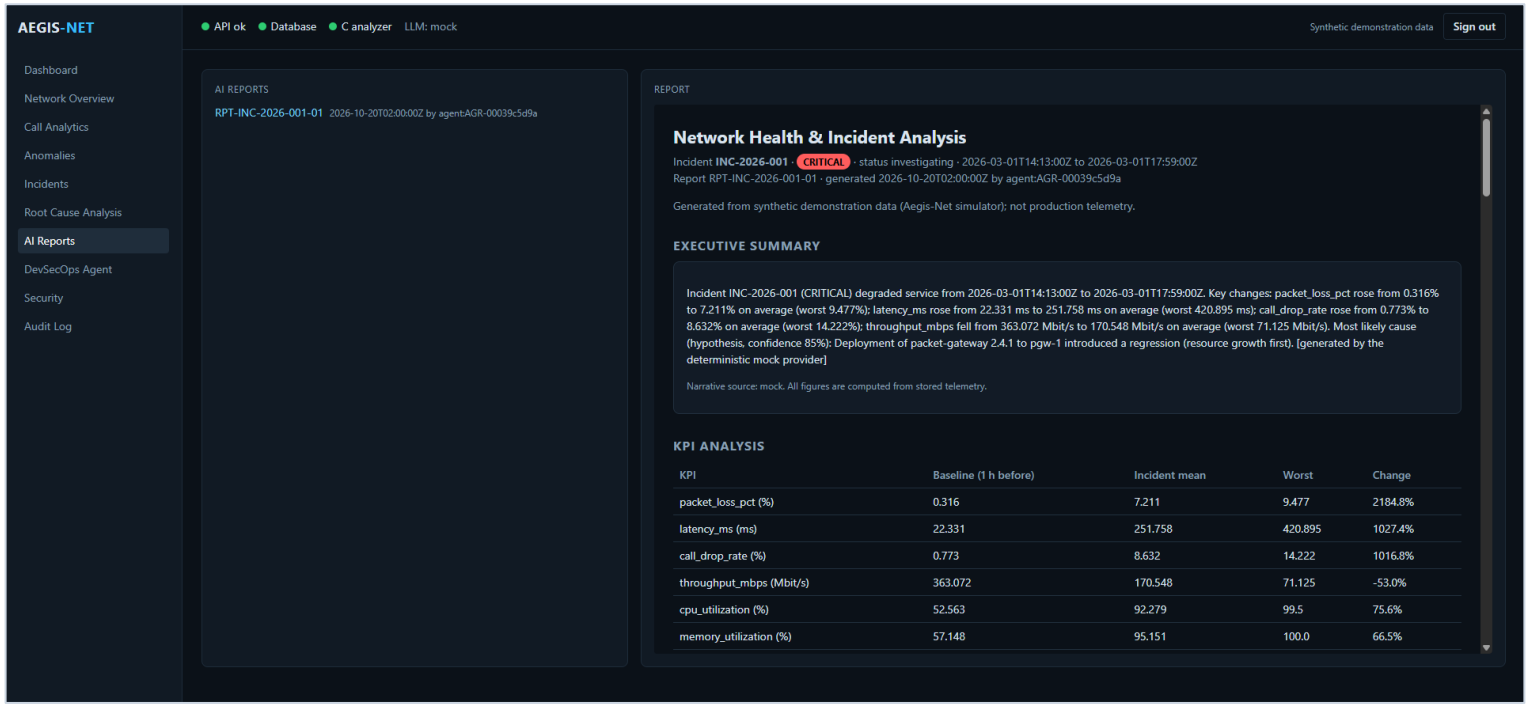
Model asks, engine decides

- 9 read-only, incident-scoped tools
- arguments validated twice (JSON schema, Pydantic)
- results capped at 16 KB; at most 10 steps, 8 calls per step
- answer: one JSON object (RCAAnswer)
- facts must cite successful tool calls
- leading hypothesis = root cause; confidence <= 0.95
- 3 attempts, then status failed; nothing invented

The mock is not an LLM. It is a rule-based investigator using the same tools and validation. Live providers were never called.

REPORTING

Reports compute every figure from stored data; only the summary is prose



Screenshot: AI Reports page (supplied document, synthetic data)

Grounded by construction

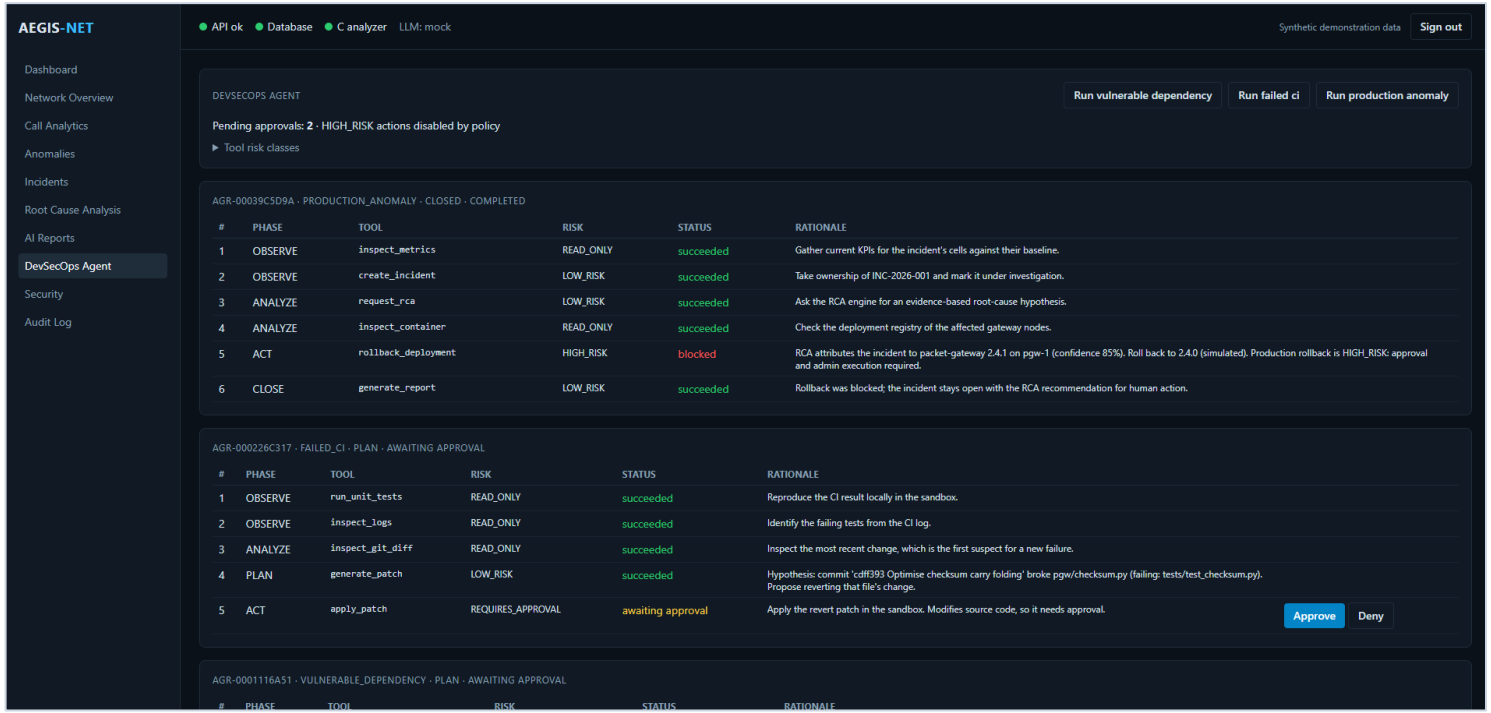
- every figure computed from the database
- baseline = mean of the hour before onset
- summary prose accepted only if all its numbers appear in the context; else template
- JSON, Markdown, HTML from one document model
- HTML autoescaped; CSP default-src 'none'; sandboxed iframe in the UI
- root cause always labelled a hypothesis

VERIFIED

PDF NOT BUILT

The agent is a deterministic, policy-controlled playbook, not an LLM planner

OBSERVE → ANALYZE → PLAN → ACT → VERIFY → CLOSE



Screenshot: DevSecOps Agent page (supplied document, synthetic data; bottom edge cropped in the source)

Three scenarios

- vulnerable dependency: scan, pin bump (approval), tests, re-scan
- failed CI: reproduce, diff, revert patch (approval), tests + linter
- production anomaly: RCA, rollback proposal (HIGH_RISK), verify

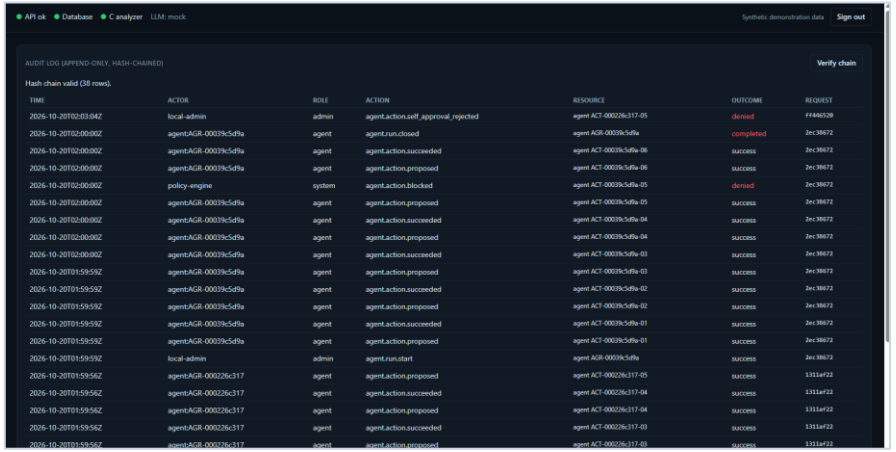
Not an LLM planner: playbooks are pure functions of earlier results; rationale stored before execution.

VERIFIED ROLLBACK SIMULATED

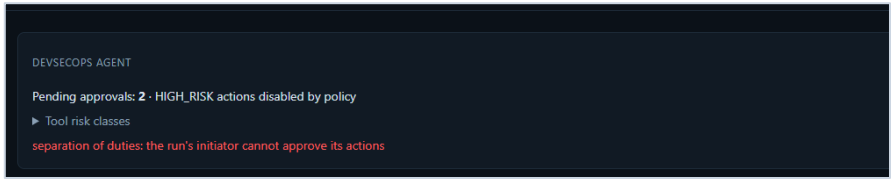
Policy is enforced in code at execution time, and every action leaves a hash-chained record

Risk class	Policy	Approval	Executed by
READ_ONLY / LOW_RISK	allow	none	agent, automatically
REQUIRES_APPROVAL	needs approval	approver, not the initiator	operator or higher
HIGH_RISK (flag off)	deny: blocked	-	nobody (human acts)
HIGH_RISK (flag on)	needs approval	approver, not the initiator	admin only
BLOCKED	deny	-	never

- policy re-checked inside execute_action, immediately before running
- separation of duties: the run's initiator cannot approve
- 20 allowlisted tools; commands: python -m pytest|pyflakes and 9 git subcommands, no shell
- audit row hash = SHA-256(content + previous hash); verify needs admin
- rewriting a whole suffix could forge the chain: external anchoring not implemented



Audit log: chain valid (38 rows); policy block and denied self-approval visible

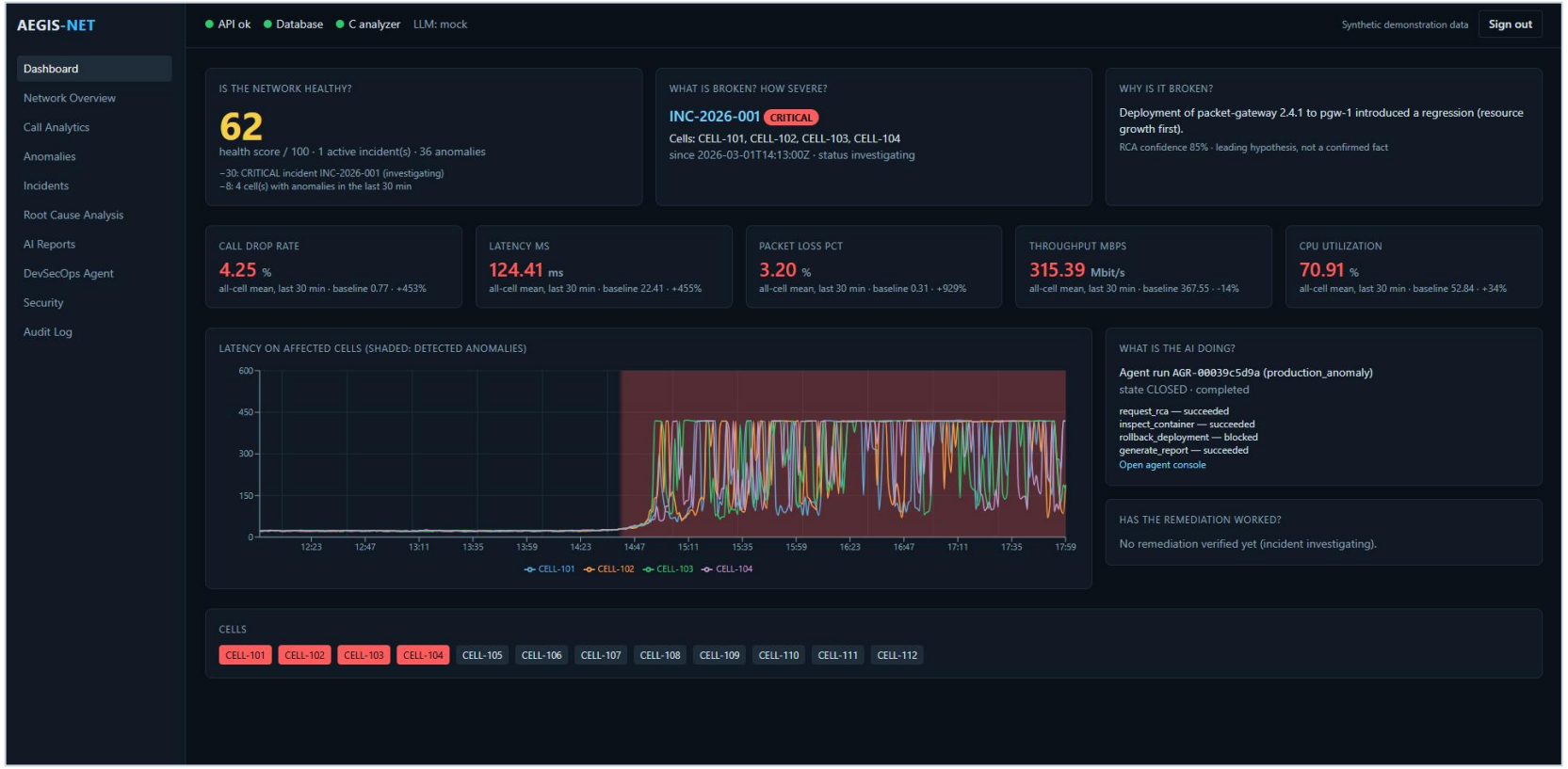


VERIFIED

36 agent tests, 22 security tests

DASHBOARD

One screen answers the six operator questions



Screenshot: Dashboard (supplied document, synthetic data)

1. **Healthy?**
   score 62 = 100 - 30 (critical) - 8 (4 cells)
2. **What, how severe?**
   INC-2026-001, CRITICAL, CELL-101..104
3. **Why?**
   RCA headline, 85%, a hypothesis
4. **KPIs and chart**
   latency on affected cells, anomaly window shaded
5. **What is the AI doing?**
   agent run and its actions
6. **Did it work?**
   not yet: incident investigating

Token in sessionStorage; roles enforced by the server.

Twenty-six routes and nineteen tables, with roles enforced per endpoint

Endpoint group	Minimum role	Routes
health	public	1
pipeline run / list	operator / viewer	2
calls, network, incidents (reads)	viewer	7
investigate (RCA), report generate, agent start	operator	3
reports, RCA, deployments, findings, agent reads	viewer	9
agent approve	approver	1
agent execute	operator (admin if HIGH_RISK)	1
audit list / verify	operator / admin	2

- bodies use extra="forbid"; ids, metrics and cells are pattern-checked
- 401 / 403 / 404 / 409 / 413 / 422 / 429 mapped; 500 carries only a request id
- every response has X-Request-ID; /metrics is not proxied

19 tables (one Alembic migration)

Run data
pipeline_runs, network_metrics, anomalies, operational_events, deployments

Call analysis
call_analysis_runs, call_cell_statistics, call_anomalies

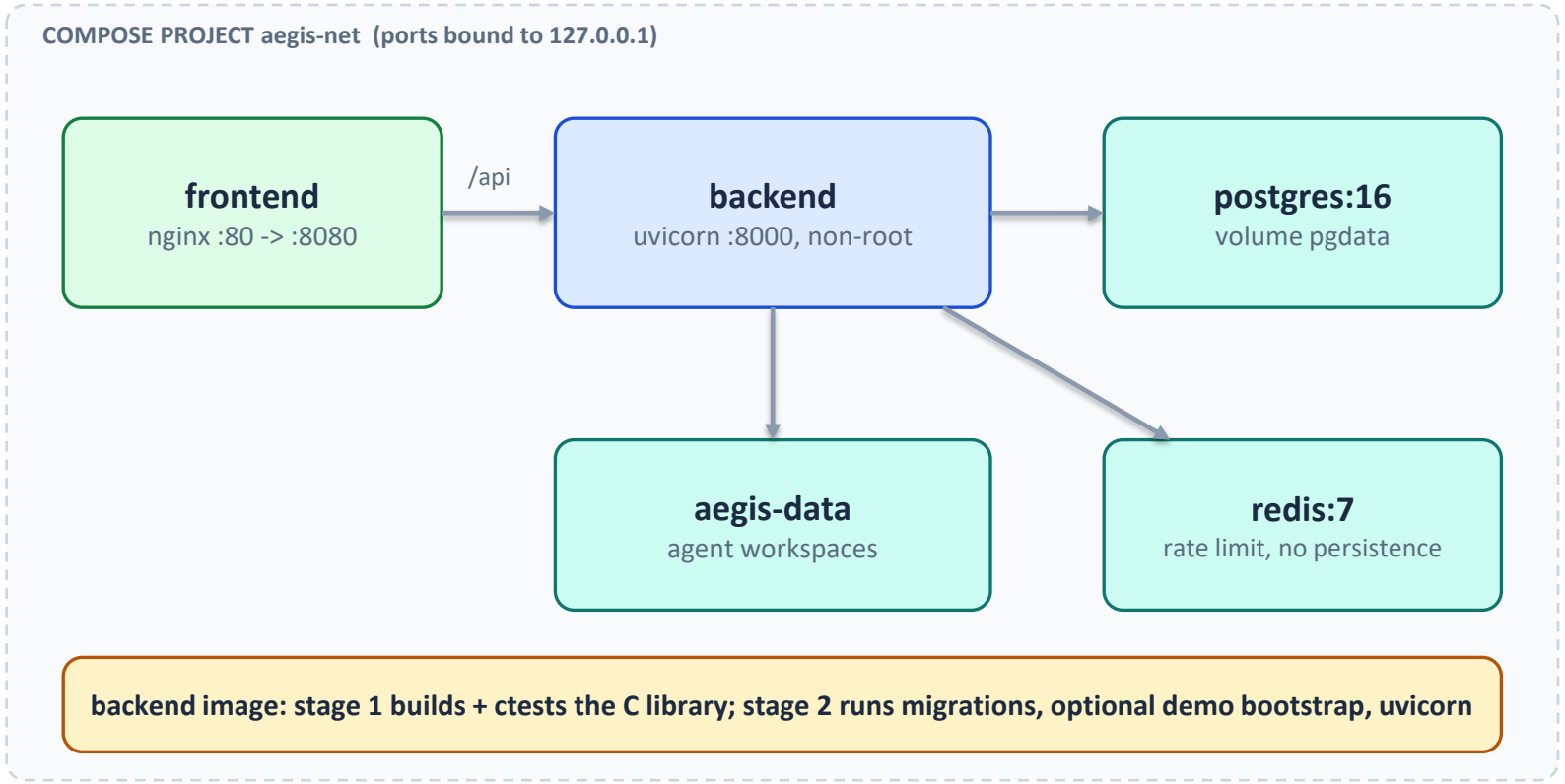
Incidents and AI
incidents, incident_cells, incident_anomalies, incident_events, root_cause_analyses, recommendations, reports

Agent and audit
agent_runs, agent_actions, security_findings, audit_logs

VERIFIED

POSTGRESQL TEST SKIPPED HERE

Four containers start with one command; the backend image builds and tests the C library



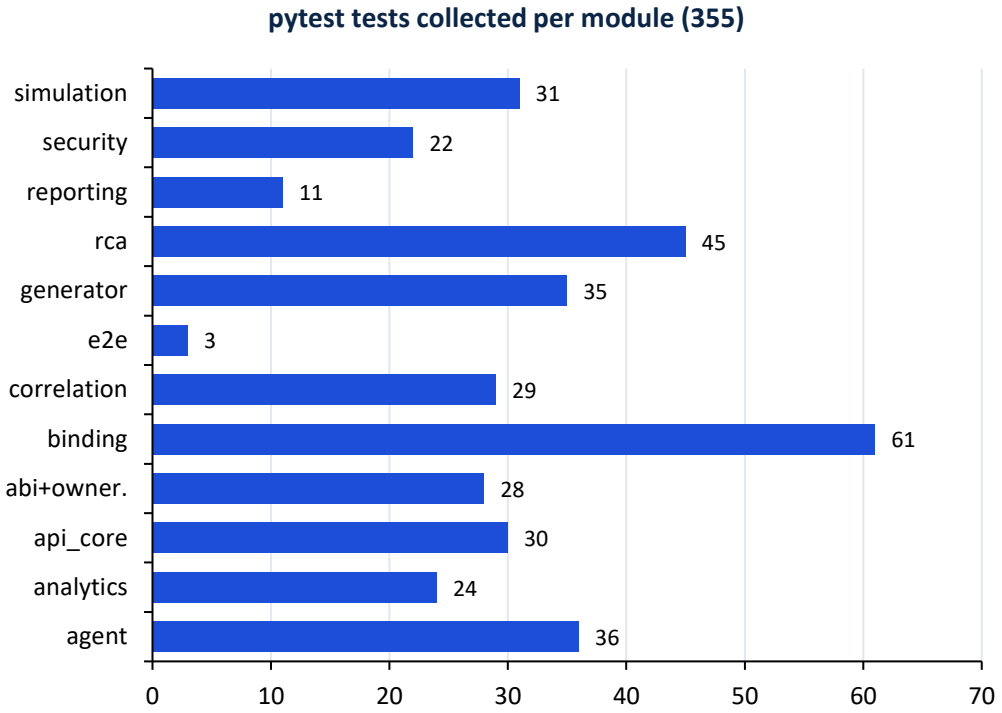

```
$ make env
  hashed tokens -> .env
$ docker compose up --build
  dashboard :8080, API docs :8000/docs
$ make docker-smoke
  isolated build + API checks, then clean-up
$ docker compose down
  keeps volumes
```

STACK NOT RE-RUN IN THIS SESSION

A clean --no-cache smoke build and four-service start are recorded in the implementation plan. Required secrets fail fast with a hint to run make env; no credentials appear in the deliverables.

Layered tests give evidence, and the gaps are stated

Layer (command)	Result run for this handbook
C tests (make c-test)	2 of 2 passed
C + ASan/UBSan/LSan + fault injection	2 of 2 passed
Python (make py-test)	352 passed, 3 skipped (207.7 s)
mypy --strict	86 files, no issues
Dashboard (make frontend-test)	tsc ok, Vitest 10/10, build ok
Offline demo (make demo)	exit 0; audit chain valid (24)
Docker smoke, ASan Python, PostgreSQL, Redis, live LLM	not run here (recorded or opt-in)



Not proven: live LLM behaviour, PostgreSQL and Redis paths (3 opt-in skips), Docker in this session, real telemetry. Simulator, thresholds and tests were written together, so results show internal consistency, not field accuracy.

LIVE DEMO

The demo runs offline in seconds; the full stack needs Docker

1	`make demo`	offline: SQLite, mock LLM, 13 steps through the real API
2	`make env + docker compose up --build`	dashboard :8080 with the seeded incident
3	**Sign in**	token from .aegis-tokens: viewer, operator, approver, admin
4	**Incidents > INC-2026-001 > Investigate**	RCA with tool-call ids (operator)
5	**AI Reports > Generate**	JSON / Markdown / HTML report (operator)
6	**DevSecOps Agent > run failed_ci**	approve with a different user, then execute
7	**Audit Log > Verify chain**	admin; read proposals, blocks, denials

No LLM key needed. The mock provider is the default; a missing key or SDK falls back to it and reports why.

STATUS

What is verified, unverified, simulated or not implemented

VERIFIED

- C library (tests + sanitizers)
- ctypes binding and ABI checks
- detectors, correlation
- API, RBAC, audit chain
- agent policy and sandbox
- offline demo, dashboard tests

UNVERIFIED HERE

- Anthropic, OpenAI, local providers (no key)
- Docker stack (recorded, not re-run)
- PostgreSQL migration test (skipped)
- live Redis test (skipped)
- py-test-asan (recorded)

SIMULATED

- all telemetry, CDRs, events
- incidents and deployments
- DEMO-CVE advisories
- production rollback
- mock provider as the "LLM"

NOT IMPLEMENTED

- PDF reports
- Isolation Forest, DBSCAN
- OpenTelemetry, Grafana
- restart_service model
- TLS, external identity, HA
- anything on real telemetry

Each item traces to a test, a command in Appendix B of the handbook, or a recorded result in IMPLEMENTATION_PLAN.md.

FUTURE WORK

Priorities for closing the largest gaps

1	**Scored live-LLM RCA evaluation**	largest unverified claim; include adversarial log lines
2	**Validate on recorded, anonymised telemetry**	measure precision and recall outside the simulator
3	**Asynchronous pipeline and agent runs**	requests return immediately; job queue
4	**Seasonal baselines + a model-based detector**	keep interpretable explanations beside it
5	**OIDC, TLS, secret manager, anchored audit head**	close the production security gaps
6	**Patches beyond reverts, real CI hooks**	validated in the sandbox
7	**OpenTelemetry traces**	API, RCA and agent

RECOMMENDATIONS ONLY

None of these exists in the repository today.

TAKEAWAYS

Evidence-grounded, policy-controlled, and honest about limits

1	**Native where it matters**	A C library does the risky parsing; tests fail allocations on purpose.
2	**Interpretable detection**	Four detectors that must agree; every cause is a hypothesis with a cap of 0.95.
3	**AI that can only read**	Allowlisted read-only tools; answers validated and cited; no hidden reasoning stored.
4	**Policy enforced in code**	Risk classes re-checked at execution; separation of duties; hash-chained audit.
5	**Honest limits**	Synthetic data, mock AI by default, simulated rollback, unverified live providers.

Start reading: call_analyzer.h, _native.py, model.py, detectors.py, correlation.py, rca/engine.py, agents/policy.py, security/audit.py